

TOPIC 35/60

Types of Tokenizers

How machines chop up language.

From microscopic characters to massive sentences, how we split text defines what a Language Model can "see" and understand. Let's explore the algorithms.

Ashish Jangra

[linkedin.com/in/ashish-jangra](https://www.linkedin.com/in/ashish-jangra)



The Three Paradigms

Before feeding text to a neural network, we must slice it into manageable pieces. There are three primary levels of granularity, each with massive trade-offs in vocabulary size and sequence length.

1

Character-Level

Splits by individual letters. (e.g., "c", "a", "t")

2

Word-Level

Splits by spaces & punctuation. (e.g., "the", "cat")

3

Subword-Level

Splits into word fragments. (e.g., "un", "belie", "able")

THE WINNER



APPROACH 1

Word-Level Tokenization

The most intuitive approach. The algorithm simply looks for spaces and punctuation, splitting the text into complete, meaningful words.

INPUT STRING

"I loved playing Minecraft."

TOKEN ARRAY

["I"

"loved"

"playing"

"Minecraft"

"."]

✓ Pros

High semantic meaning. Every token represents a complete concept. Very short sequence lengths.

✗ Cons

Infinite Vocabulary. Fails completely on typos, combined words, or morphs (e.g., *play*, *plays*, *playing* are treated as 3 entirely unrelated vectors).

Ashish Jangra

[linkedin.com/in/ashish-jangra](https://www.linkedin.com/in/ashish-jangra)



APPROACH 2

Character-Level Tokenization

The exact opposite of Word-Level. We treat every individual letter, number, and symbol as its own unique token.

INPUT STRING

"Minecraft"

TOKEN ARRAY

["M" "i" "n" "e" "c" "r" "a" "f" "t"]

✓ Pros

Tiny vocabulary (e.g., ~256 tokens). Zero Out-Of-Vocabulary (OOV) errors. Can spell literally anything.

✗ Cons

Context Exhaustion. "M" has no semantic meaning alone. It takes 9 tokens just to say one word, blowing up API costs and destroying the context window.

Ashish Jangra

[linkedin.com/in/ashish-jangra](https://www.linkedin.com/in/ashish-jangra)



THE OUTLIER

Sentence-Level Tokenization

While LLMs don't generate text sentence-by-sentence, sentence tokenization is highly prevalent in the data prep phase (like Retrieval-Augmented Generation / RAG).

Text Splitting for Vector Databases

Tools like NLTK or SpaCy use grammar rules (periods, question marks) to break large PDFs into complete thoughts.

Chunk 1: "The company reported record profits in Q3."

Chunk 2: "This was driven largely by AI infrastructure sales."

Verdict: Perfect for document chunking. Terrible for neural network embedding tables (the vocabulary would essentially be infinite).

Ashish Jangra

[linkedin.com/in/ashish-jangra](https://www.linkedin.com/in/ashish-jangra)



THE GOLDILOCKS ZONE

Subword Tokenization

Modern LLMs (GPT, Llama, Claude) use a hybrid approach. Frequently used words are kept whole, while rare words are chopped into phonetic or structural "Lego blocks."

COMMON WORD (KEPT WHOLE)

"Apple" → ["Apple"]

RARE WORD (SHATTERED INTO SUBWORDS)

"Unbelievably"

["Un"] + "believ" + "ably"]

The model reuses "Un" and "ably" for thousands of other words, saving vocabulary space.

Ashish Jangra

[linkedin.com/in/ashish-jangra](https://www.linkedin.com/in/ashish-jangra)



ALGORITHM 1

Byte-Pair Encoding (BPE)

Used by OpenAI (GPT-4) and Meta (Llama 3). BPE is a statistical, frequency-based compression algorithm. It finds the most frequently adjacent pairs of characters and merges them.

Training the Tokenizer:

1. Start with a vocabulary of pure individual bytes/characters.
2. Scan the training text and count adjacent pairs (e.g., `"e" + "s" = "es"` occurs 1M times).
3. Merge the most frequent pair into a new single token: `"es"`.
4. Repeat step 2 and 3 until you hit your target vocabulary size (e.g., 100,000 tokens).

Result: `"es"`, `"ing"`, `"tion"`, and common full words become single tokens because they appear frequently in English web data.

Ashish Jangra

[linkedin.com/in/ashish-jangra](https://www.linkedin.com/in/ashish-jangra)



ALGORITHM 2

WordPiece (BERT)

Developed by Google to train BERT. It is very similar to BPE, but instead of merging based on raw frequency, it merges based on **likelihood** (which pair most increases the probability of the training data).

The "##" Prefix

WordPiece visually tags subwords that are attached to the end of a previous word using double hashes.

Input:

"playing"

WordPiece Output:

["play",

"##ing"]

Ashish Jangra

[linkedin.com/in/ashish-jangra](https://www.linkedin.com/in/ashish-jangra)



ALGORITHM 3

Unigram & SentencePiece

Used by Google's T5 and ALBERT. Instead of building up from characters, Unigram starts with a massive, bloated vocabulary and mathematically **prunes** away the least useful tokens until it hits the target size.

SentencePiece: The Meta-Tokenizer

SentencePiece isn't an algorithm itself, but a library that implements BPE or Unigram **without relying on spaces**.

Treating spaces as characters:

"Hello world" → ["_Hello", "_world"]

Crucial for languages like Chinese or Japanese that do not use spaces between words.

Ashish Jangra

[linkedin.com/in/ashish-jangra](https://www.linkedin.com/in/ashish-jangra)



HOW MODELS CHOP DATA.

CONNECT WITH ME



[linkedin.com/in/ashish-jangra](https://www.linkedin.com/in/ashish-jangra)

